

Question-5.4.17

EE25BTECH11022 - sankeerthan

Question

Using elementary transformations, find the inverse of the following matrix.

$$\begin{pmatrix} 3 & 1 \\ 5 & 2 \end{pmatrix}$$

Solution

Given

$$\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 5 & 2 \end{pmatrix} \quad (1)$$

Let $\mathbf{A}^{-1}$ be the inverse of $\mathbf{A}$. Then

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{I} \quad (2)$$

Augmented matrix of $(\mathbf{A} \mid \mathbf{I})$ is given by

$$\left(\begin{array}{cc|cc} 3 & 1 & 1 & 0 \\ 5 & 2 & 0 & 1 \end{array} \right) \xrightarrow{R_2 \rightarrow 3R_2 - 5R_1} \left(\begin{array}{cc|cc} 3 & 1 & 1 & 0 \\ 0 & 1 & -5 & 3 \end{array} \right) \xrightarrow{R_1 \rightarrow R_1 - R_2} \left(\begin{array}{cc|cc} 3 & 0 & 6 & -3 \\ 0 & 1 & -5 & 3 \end{array} \right) \quad (3)$$

$$\left(\begin{array}{cc|cc} 3 & 0 & 6 & -3 \\ 0 & 1 & -5 & 3 \end{array} \right) \xrightarrow{R_1 \rightarrow \frac{1}{3}R_1} \left(\begin{array}{cc|cc} 1 & 0 & 2 & -1 \\ 0 & 1 & -5 & 3 \end{array} \right) \quad (4)$$

Hence the inverse of the matrix $\begin{pmatrix} 3 & 1 \\ 5 & 2 \end{pmatrix}$ is $\begin{pmatrix} 2 & -1 \\ -5 & 3 \end{pmatrix}$

```
// code.c
#include <stdio.h>

// Function to multiply matrix and vector
void mat_vec_mult(double mat[2][2], double vec[2], double result
    [2]) {
    for (int i = 0; i < 2; ++i) {
        result[i] = 0;
        for (int j = 0; j < 2; ++j) {
            result[i] += mat[i][j] * vec[j];
        }
    }
}
```

```
// Solves the equation  $X = A_{inv} * B$  where  $A_{inv}$  is the inverse  
of the given matrix  
void find_points(double x_vals[2], double y_vals[2], double  
    points[2][2]) {  
    // Inverse of given matrix A [[3, 1], [5, 2]] is [[2, -1],  
        [-5, 3]]  
    double inv[2][2] = {{2, -1}, {-5, 3}};  
    for (int i = 0; i < 2; ++i) {  
        double b[2] = {x_vals[i], y_vals[i]};  
        double res[2];  
        mat_vec_mult(inv, b, res);  
        points[i][0] = res[0];  
        points[i][1] = res[1];  
    }  
}
```

```
// Main just prints out by default
int main() {
    double x_vals[2] = {1, 4}; // Example points on x
    double y_vals[2] = {1, 4}; // Example points on y
    double points[2][2];
    find_points(x_vals, y_vals, points);
    for (int i = 0; i < 2; ++i) {
        printf("%lf %lf\n", points[i][0], points[i][1]);
    }
    return 0;
}
```

```
# Code by GVV Sharma  
# Modified for Problem Solution  
# Released under GNU GPL  
# Calculating area enclosed between curves  
# call.py  
import ctypes  
import numpy as np  
import os  
  
so_file = os.path.abspath('./code.so')  
lib = ctypes.CDLL(so_file)
```

```
# define C double arrays
class DoubleArray2(ctypes.Structure):
    _fields_ = [("arr", ctypes.c_double * 2)]
def create_double2(np_array):
    array_type = ctypes.c_double * 2
    return array_type(*np_array)

# C function
double2 = ctypes.c_double * 2
points_type = double2 * 2

find_points = lib.find_points
find_points.argtypes = [double2, double2, points_type]
find_points.restype = None
```



```
def get_points(x_vals, y_vals):
    x_c = double2(*x_vals)
    y_c = double2(*y_vals)
    out = points_type()
    find_points(x_c, y_c, out)
    result = np.array([[out[i][0], out[i][1]] for i in range(2)])
    return result

if __name__ == '__main__':
    x_vals = [1.0, 4.0]
    y_vals = [1.0, 4.0]
    pts = get_points(x_vals, y_vals)
    print(pts)
```